

PAG 融合设计师规范

一、 文字替换

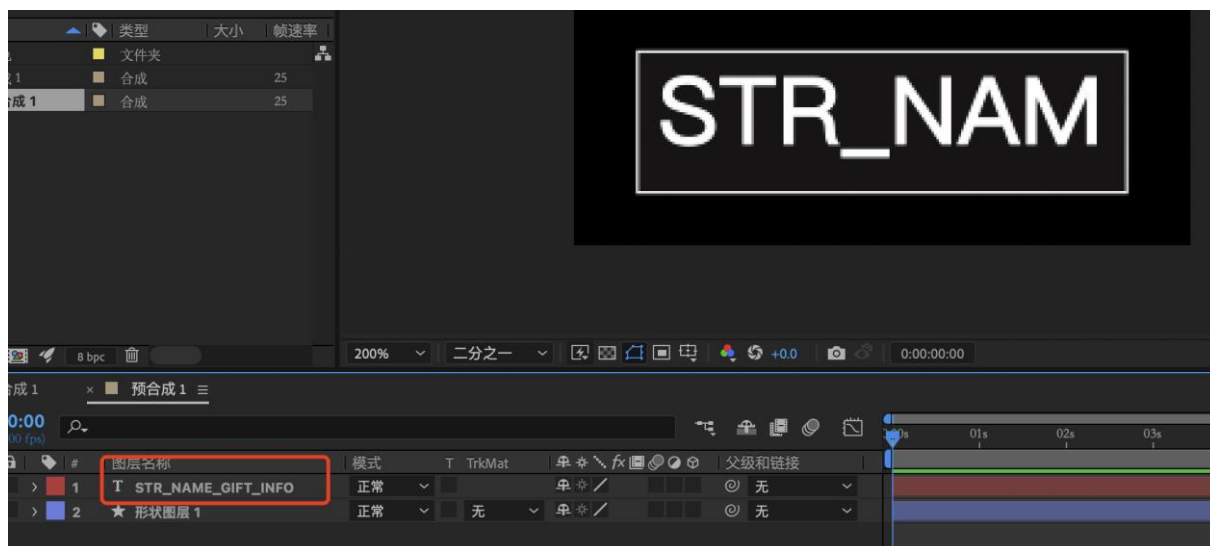
注意：以下文字均使用 box 文字；如下图，文本背景为黑色，沿着背景画一个 box 文本区域，可以保证文字在此区域展示：



如果文本想在客户端替换为指定文字，目前客户端预埋了以下文字可供替换：

- STR_NICK_NAME_GIFT_FROM：送礼人的昵称
- STR_NICK_NAME_GIFT_TO：收礼人的昵称
- STR_NAME_GIFT_INFO：礼物名称
- INT_NUM_GIFT：礼物数量
- INT_ROOM_ID：房间 ID

在设计时，将需要替换的地方改为以上英文即可，比如上面的横幅，我想替换为礼物名称，即为：



如果超长，则会默认以...结尾（入端效果）



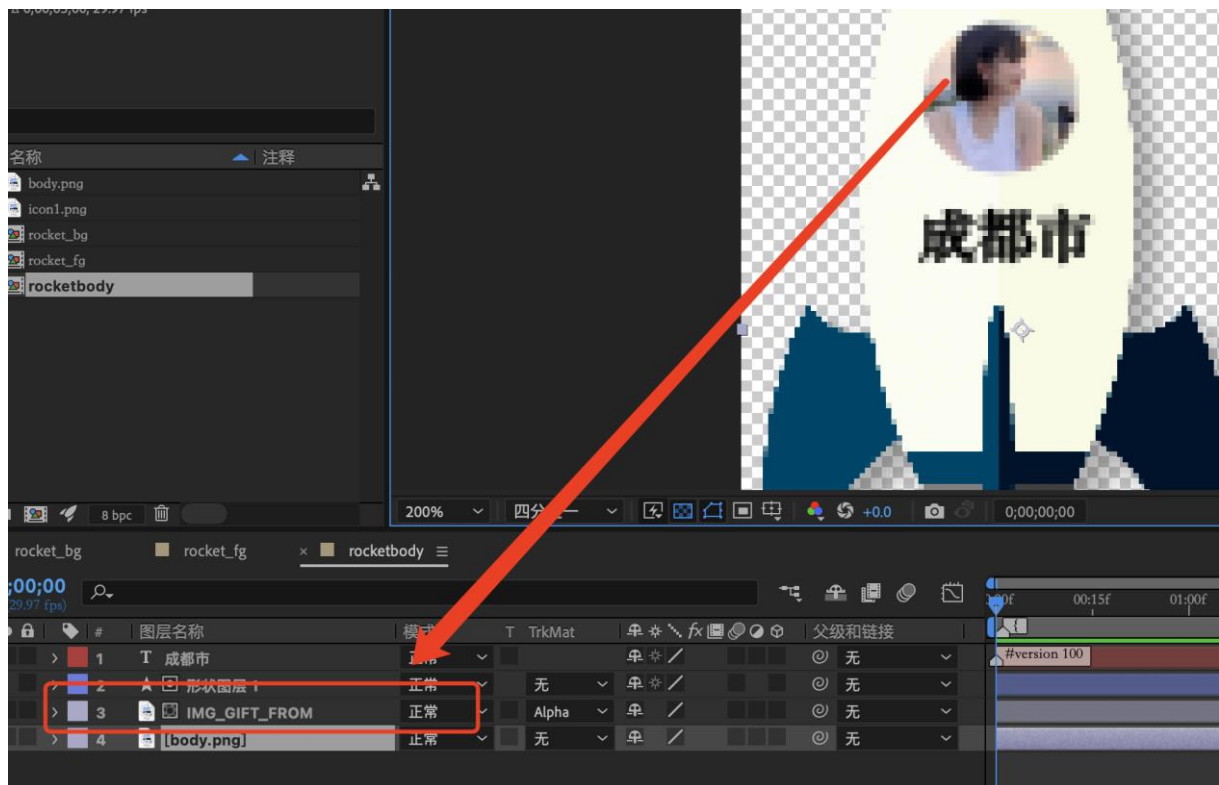
如果 box 区域较大，但文字字体较小的情况，会分行：



二、 头像替换

头像需要一个默认 png 结尾的图片作为占位符，客户端预埋了以下图片可作为替换：

- IMG_GIFT_FROM：送礼人的头像
- IMG_GIFT_TO：收礼人的头像
- IMG_GIFT_INFO：礼物图标



把需要替换的图标改为对应上述英文即可（不要 png 后缀）

注意：

- 头像要放一张默认图(图片上不能有 app 标识)，不能留黑或留白
- 礼物图标要放一张默认图((图片上不能有 app 标识)，不能留黑或留白

三、 滤镜支持

使用后处理锐化滤镜可提升清晰度，对比效果如下：



具体做法为：将项目打包为一个预合成，给此预合成加上” 标记 “marker，填入锐化代码即可：

```

#version 100

precision mediump float;
varying highp vec2 vertexColor;

uniform float uWidth;
uniform float uHeight;
uniform sampler2D inputImageTexture;

void main() {
    vec2 uv = vertexColor;

    vec4 color = texture2D(inputImageTexture, uv);

    float sharpness = 0.3;

    float xOffset = 1. / uWidth;
    float yOffset = 1. / uHeight;

    vec4 neighbors[4];
    neighbors[0] = texture2D(inputImageTexture, uv + vec2(-
xOffset, -yOffset));
    neighbors[1] = texture2D(inputImageTexture, uv +
vec2(xOffset, yOffset));
    neighbors[2] = texture2D(inputImageTexture, uv +
vec2(xOffset, -yOffset));
    neighbors[3] = texture2D(inputImageTexture, uv + vec2(-
xOffset, yOffset));

    vec4 sharpenedColor = color * (sharpness * 4.0 + 1.0);

    sharpenedColor -= neighbors[0] * sharpness;
    sharpenedColor -= neighbors[1] * sharpness;
    sharpenedColor -= neighbors[2] * sharpness;
    sharpenedColor -= neighbors[3] * sharpness;

    gl_FragColor = sharpenedColor;
}

```



在导出时，pag 会有提示，可忽略



该滤镜其实可以应用到任何图层（但要保证不是 bmp 的子图层），可按照实际情况使用；